brings us back to early and continuous delivery through good architectures, designs, and technical excellence.

The previous chapters give you the tools you need to measure functional and non-functional requirements. This chapter uses the tools and data we've explored up to this point to measure the technical quality of your software products.

If you drill into the agile principles from the Agile Manifesto, you'll see a strong focus on delivering frequently in the face of change:

- Your highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference for the shorter timescale.

All of these are telling you to change things quickly and frequently. Then there are a few that go even deeper:

- Continuous attention to technical excellence and good design enhances agility.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- Working software is the primary measure of progress.

Anyone with an IDE and access to the internet can create working software; the really important thing to dig into is how to create a software product that you can iterate on quickly and is stable enough to handle your consumer's expectations. At a glance, measuring a codebase across these principles looks a bit nebulous, so to do that we'll dive into non-functional requirements, or the code "ilities."

Measuring your team against all the agile tenets